

Name: Javaughn Stevens

CMSC 204 Spotify Project Project 3 Writeup

1. What did you learn?

This project helped me strengthen my understanding of nodes and LinkedList and how it differs when compared to ArrayList and array-based implementations of projects I have done thus far in CMSC 204. I also revisited older knowledge such as using StringBuilder, reading/writing to txt files, and other important, underutilized concepts from CMSC 203.

2. What issues did you encounter, if any?

I ran into some issues with understanding how throwing specific exceptions in method headers (such as throwing UserNotFoundException vs just throwing Exception which acts as a catch-all for exceptions). Normally I like to use catch-all for things like exceptions or imports but I have seen in this project where that is not always the best decision.

3. What would you have done differently?

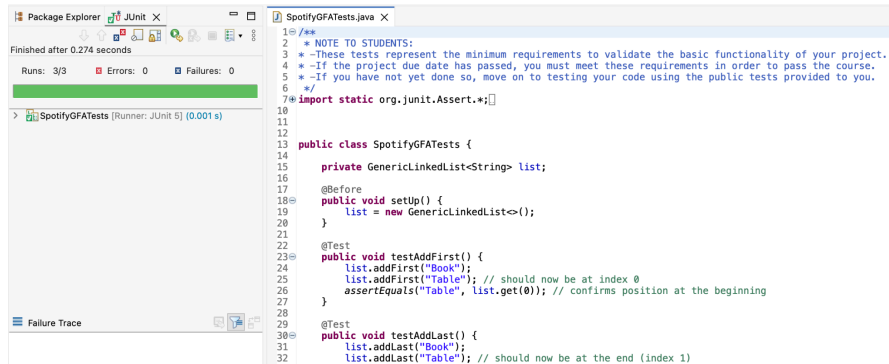
I would spend more time reading and practicing the concepts from the text before beginning implementation of a project because I never know what aspects of Java could be useful in addition to the weekly slides and videos.

4. How can you apply this concept in the future?

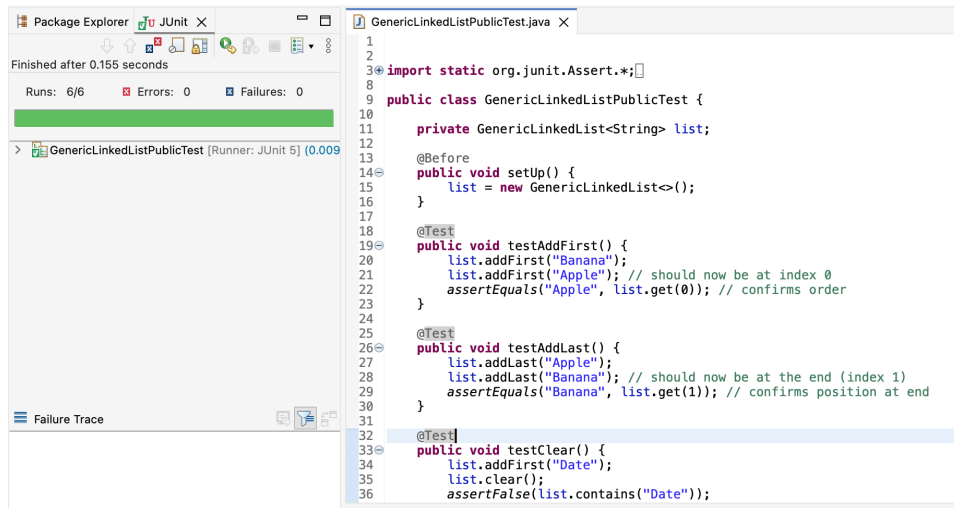
This concept can be applied to a variety of real-world websites where users can register and manage groups of something, where it be programming projects in GitHub or managing a music playlist, spreadsheets in Excel, and similar things.

JUnit Test Screenshots: (note, I included Javadoc comments afterwards)

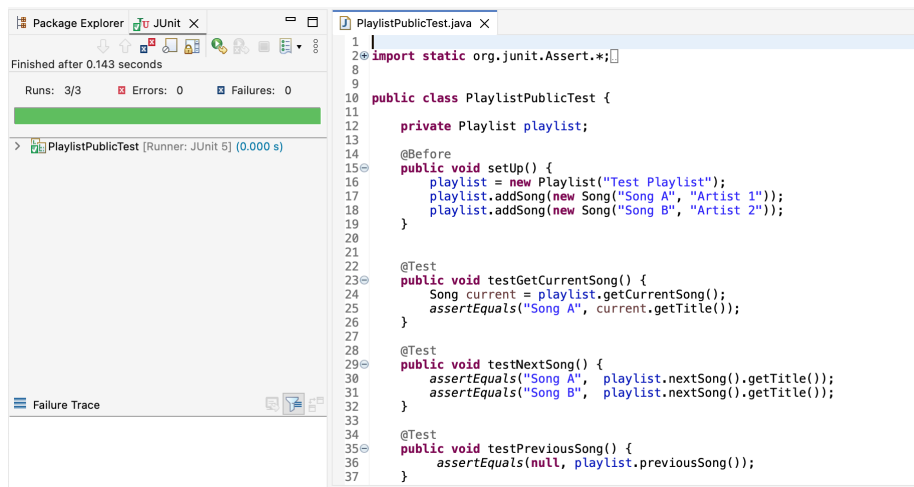
SpotifyGFATests.java



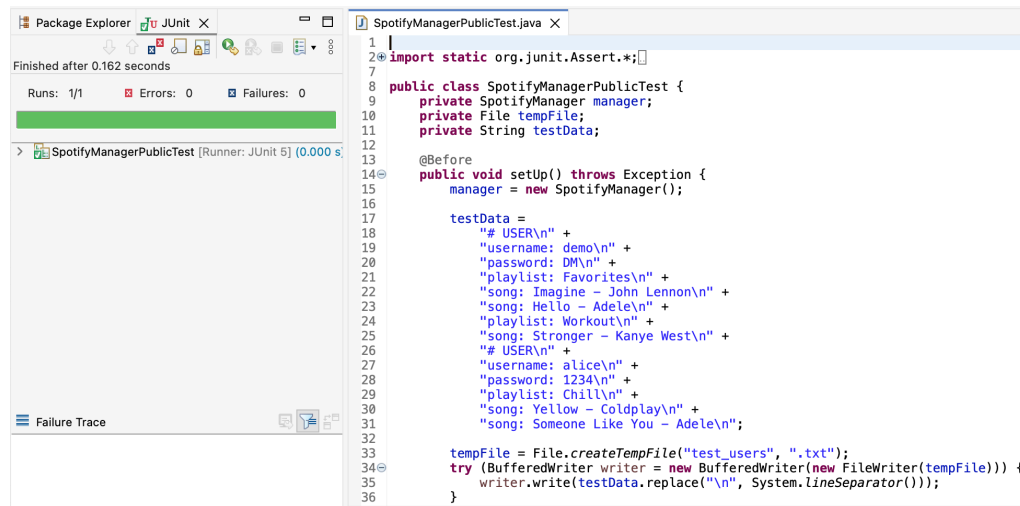
GenericLinkedListPublicTest.java



PlaylistPublicTest.java

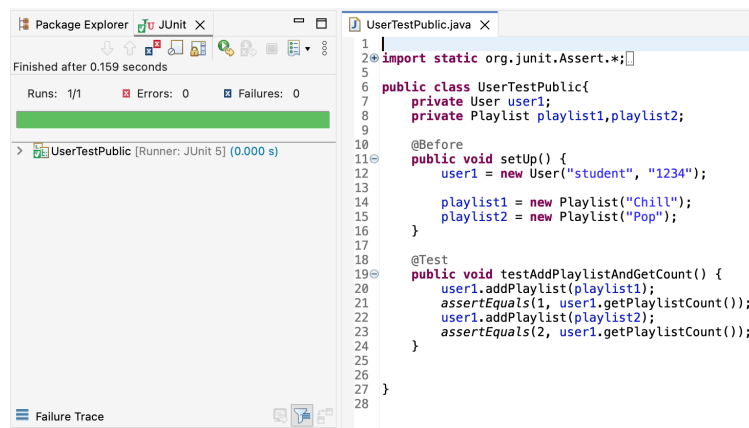


SpotifyManagerPublicTest.java



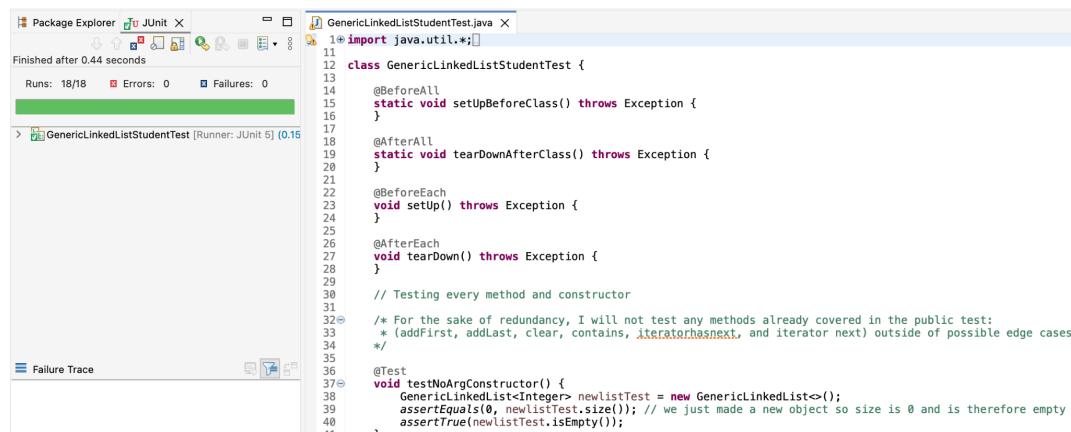
```
1 |
2 | import static org.junit.Assert.*;
3 |
4 |
5 |
6 |
7 |
8 | public class SpotifyManagerPublicTest {
9 |     private SpotifyManager manager;
10 |    private File tempFile;
11 |    private String testData;
12 |
13 |
14 |    @Before
15 |    public void setUp() throws Exception {
16 |        manager = new SpotifyManager();
17 |
18 |        testData =
19 |            "# USER\n" +
20 |            "username: demo\n" +
21 |            "password: DM\n" +
22 |            "playlist: Favorites\n" +
23 |            "song: Imagine - John Lennon\n" +
24 |            "song: Hello - Adele\n" +
25 |            "playlist: Workout\n" +
26 |            "song: Stronger - Kanye West\n" +
27 |            "# USER\n" +
28 |            "username: alice\n" +
29 |            "password: 1234\n" +
30 |            "playlist: Chill\n" +
31 |            "song: Yellow - Coldplay\n" +
32 |            "song: Someone Like You - Adele\n";
33 |
34 |        tempFile = File.createTempFile("test_users", ".txt");
35 |        try (BufferedWriter writer = new BufferedWriter(new FileWriter(tempFile))) {
36 |            writer.write(testData.replace("\n", System.lineSeparator()));
37 |        }
38 |    }
39 |
40 |
41 |
42 |
43 |
44 |
45 |
46 |
47 |
48 |
49 |
50 |
51 |
52 |
53 |
54 |
55 |
56 |
57 |
58 |
59 |
60 |
61 |
62 |
63 |
64 |
65 |
66 |
67 |
68 |
69 |
70 |
71 |
72 |
73 |
74 |
75 |
76 |
77 |
78 |
79 |
80 |
81 |
82 |
83 |
84 |
85 |
86 |
87 |
88 |
89 |
90 |
91 |
92 |
93 |
94 |
95 |
96 |
97 |
98 |
99 |
100 |
```

UserTestPublic.java



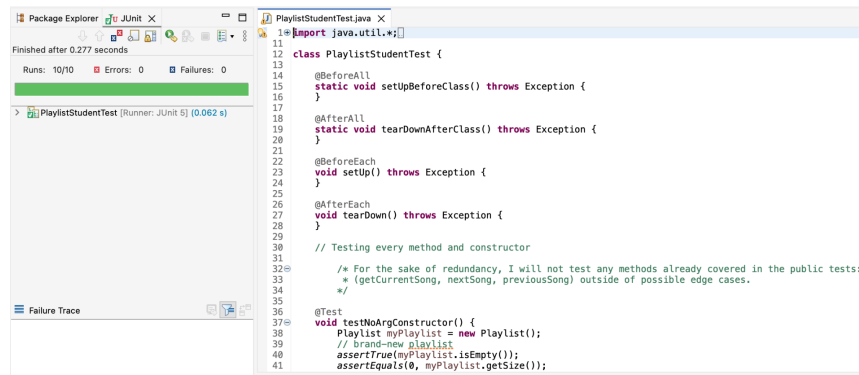
```
1 |
2 | import static org.junit.Assert.*;
3 |
4 |
5 |
6 | public class UserTestPublic {
7 |     private User user1;
8 |     private Playlist playlist1, playlist2;
9 |
10 |
11 |    @Before
12 |    public void setUp() {
13 |        user1 = new User("student", "1234");
14 |
15 |        playlist1 = new Playlist("Chill");
16 |        playlist2 = new Playlist("Pop");
17 |    }
18 |
19 |    @Test
20 |    public void testAddPlaylistAndGetCount() {
21 |        user1.addPlaylist(playlist1);
22 |        assertEquals(1, user1.getPlaylistCount());
23 |        user1.addPlaylist(playlist2);
24 |        assertEquals(2, user1.getPlaylistCount());
25 |    }
26 |
27 |
28 |
29 |
30 |
31 |
32 |
33 |
34 |
35 |
36 |
37 |
38 |
39 |
40 |
41 |
42 |
43 |
44 |
45 |
46 |
47 |
48 |
49 |
50 |
51 |
52 |
53 |
54 |
55 |
56 |
57 |
58 |
59 |
60 |
61 |
62 |
63 |
64 |
65 |
66 |
67 |
68 |
69 |
70 |
71 |
72 |
73 |
74 |
75 |
76 |
77 |
78 |
79 |
80 |
81 |
82 |
83 |
84 |
85 |
86 |
87 |
88 |
89 |
90 |
91 |
92 |
93 |
94 |
95 |
96 |
97 |
98 |
99 |
100 |
```

GenericLinkedListStudentTest.java



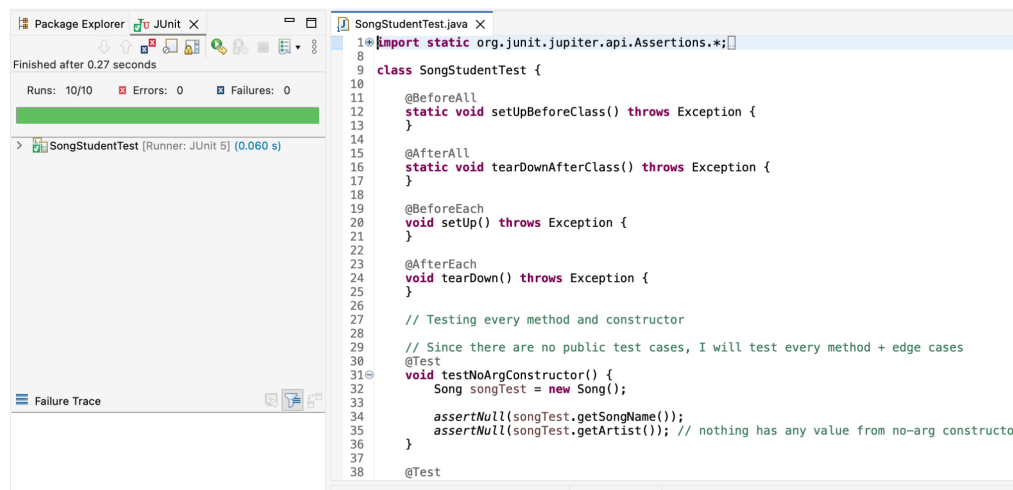
```
1 | import java.util.*;
2 |
3 |
4 |
5 |
6 |
7 |
8 |
9 |
10 |
11 |
12 |
13 |
14 |
15 |
16 |
17 |
18 |
19 |
20 |
21 |
22 |
23 |
24 |
25 |
26 |
27 |
28 |
29 |
30 |
31 |
32 |
33 |
34 |
35 |
36 |
37 |
38 |
39 |
40 |
41 |
42 |
43 |
44 |
45 |
46 |
47 |
48 |
49 |
50 |
51 |
52 |
53 |
54 |
55 |
56 |
57 |
58 |
59 |
60 |
61 |
62 |
63 |
64 |
65 |
66 |
67 |
68 |
69 |
70 |
71 |
72 |
73 |
74 |
75 |
76 |
77 |
78 |
79 |
80 |
81 |
82 |
83 |
84 |
85 |
86 |
87 |
88 |
89 |
90 |
91 |
92 |
93 |
94 |
95 |
96 |
97 |
98 |
99 |
100 |
```

PlaylistStudentTest.java



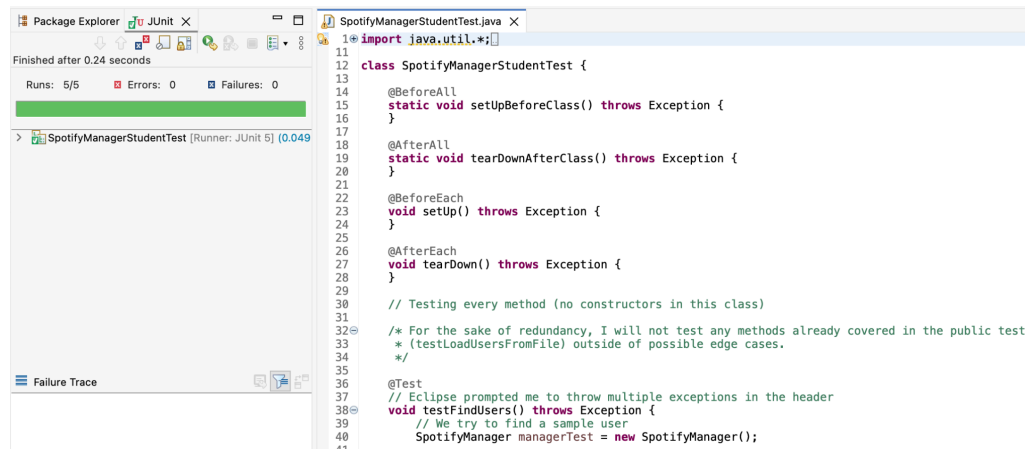
```
1 import java.util.*;
2
3 class PlaylistStudentTest {
4
5     @BeforeAll
6     static void setUpBeforeClass() throws Exception {
7     }
8
9     @AfterAll
10    static void tearDownAfterClass() throws Exception {
11    }
12
13    @BeforeEach
14    void setUp() throws Exception {
15    }
16
17    @AfterEach
18    void tearDown() throws Exception {
19    }
20
21    // Testing every method and constructor
22
23    /* For the sake of redundancy, I will not test any methods already covered in the public tests:
24     * (getCurrentSong, nextSong, previousSong) outside of possible edge cases.
25     */
26
27    @Test
28    void testNoArgConstructor() {
29        Playlist myPlaylist = new Playlist();
30        // brand-new playlist
31        assertTrue(myPlaylist.isEmpty());
32        assertEquals(0, myPlaylist.getSize());
33    }
34
35 }
```

SongStudentTest.java



```
1 import static org.junit.jupiter.api.Assertions.*;
2
3 class SongStudentTest {
4
5     @BeforeAll
6     static void setUpBeforeClass() throws Exception {
7     }
8
9     @AfterAll
10    static void tearDownAfterClass() throws Exception {
11    }
12
13    @BeforeEach
14    void setUp() throws Exception {
15    }
16
17    @AfterEach
18    void tearDown() throws Exception {
19    }
20
21    // Testing every method and constructor
22
23    // Since there are no public test cases, I will test every method + edge cases
24
25    @Test
26    void testNoArgConstructor() {
27        Song songTest = new Song();
28
29        assertNull(songTest.getSongName());
30        assertNull(songTest.getArtist()); // nothing has any value from no-arg constructor
31    }
32
33    @Test
34
35 }
```

SpotifyManagerStudentTest.java



```
1 import java.util.*;
2
3 class SpotifyManagerStudentTest {
4
5     @BeforeAll
6     static void setUpBeforeClass() throws Exception {
7     }
8
9     @AfterAll
10    static void tearDownAfterClass() throws Exception {
11    }
12
13    @BeforeEach
14    void setUp() throws Exception {
15    }
16
17    @AfterEach
18    void tearDown() throws Exception {
19    }
20
21    // Testing every method (no constructors in this class)
22
23    /* For the sake of redundancy, I will not test any methods already covered in the public test:
24     * (testLoadUsersFromFile) outside of possible edge cases.
25     */
26
27    @Test
28    // Eclipse prompted me to throw multiple exceptions in the header
29    void testFindUsers() throws Exception {
30        // We try to find a sample user
31        SpotifyManager managerTest = new SpotifyManager();
32    }
33
34 }
```

GUI Screenshots:

